Tu trabajo dentro del proyecto

Maki, tu responsabilidad: identificar problemas de calidad y tratar faltantes, repetidos y valores extremos sin ocultar lo que cambió.

¿Qué construimos entre los tres?

Un pipeline ETL es una secuencia de pasos para extraer datos, transformarlos y cargarlos en una base. Aquí recibe registros de cajeros ATM y transferencias digitales. Un registro es una fila u objeto que describe una operación; un lote es el conjunto que procesamos en una ejecución.

El sistema mide problemas, aplica reglas, guarda aceptadas en **transacciones_limpias** y conserva inválidas en **cuarentena_errores**. Omitir un duplicado significa registrar que ya había una copia, no perderlo sin explicación.

Rol	Qué entrega al equipo
Maki / ingeniería 1	Diagnóstico, faltantes, duplicados e IQR.
Avilix / ingeniería 2	Lectura, integración, divisas, catálogo, MinMax y conexión del flujo.
Leo / ingeniería 3	Modelos Pydantic, errores detallados, persistencia y pruebas.

Archivos que debes reconocer

src/calidad.py: perfilar y comparar_calidad.
src/duplicados.py: separar_exactos y detectar_conflictos.
src/limpieza.py: imputar y aplicar_iqr.
Pruebas: tests/test_calidad.py y tests/test_limpieza.py.

> Esta guía explica el ETL obligatorio de los puntos 4 y 6 del profesor. No necesitas explicar dashboard ni simuladores. Los módulos se integraron y adaptaron en conjunto: el rol indica responsabilidad técnica, no autoría exclusiva de cada línea.

1. Medir antes de limpiar

perfilar recorre los registros crudos sin modificarlos. Cuenta nulos, cadenas vacías, tipos observados y copias exactas. Un nulo es ausencia de valor; una cadena vacía es texto sin contenido o solo espacios.

Medición real	ATM	DIGITAL
Filas leídas	620	600
Duplicados exactos adicionales	20	0
Monto nulo	27	0
Identificador de cajero nulo	44	No aplica
Correo nulo / vacío	No aplica	8 / 8

Monto ATM: 27 / 620 × 100 = **4.35 %** de nulos. Cajero: 44 / 620 × 100 = **7.10 %**. Se calculan antes de retirar duplicados o rechazar filas. Los tipos observados son tipos Python como str, int o datetime, no una inferencia de esquema de pandas.

Qué hacemos con duplicados

separar_exactos serializa cada registro con las claves ordenadas. Conserva la primera aparición y registra las posiciones adicionales. Cambiar el orden de las claves JSON no convierte el contenido en una operación distinta.

Una clave repetida con diferente contenido es un conflicto. El pipeline integrado agrupa por ID después de normalizar y compara huellas. Si hay contradicción, rechaza el conflicto. detectar_conflictos existe en tu módulo, pero el orquestador usa su propia comparación de huellas normalizadas.

Antes y después

ATM termina con 568 limpias, 32 rechazadas y 20 omitidas. DIGITAL termina con 504 limpias y 96 rechazadas. Antes y después son poblaciones distintas: no atribuyas toda disminución de nulos a imputación.

> Digital termina con 13 correos nulos en aceptadas. El correo es opcional y los vacíos se convierten a null. Una tabla limpia puede contener nulos permitidos. Limpieza no significa rellenarlo todo.

2. Entender la imputación

Imputar es sustituir una ausencia con una estimación o etiqueta definida. No recupera automáticamente el dato verdadero. Texto corrupto o un monto negativo no se tratan como faltantes que podamos rellenar.

MCAR y MAR

MCAR: la ausencia no depende de las variables del conjunto. **MAR:** puede relacionarse con variables observadas, como operación o estado. Agrupamos bajo una hipótesis MAR operativa. El perfil descriptivo no demuestra el mecanismo de ausencia ni su causa.

Campo	Grupo usado para la mediana
Monto ATM	Fuente + operación + estado de operación
Monto digital, en MXN	Fuente + moneda original + estado de operación
Comisión ATM	Fuente + operación + cajero conocido

Se exigen al menos **cinco valores observados válidos**. Se guardan mediana, media y tamaño. Si el grupo no alcanza, el motivo es SIN_BASE_PARA_IMPUTAR. Cinco es un mínimo operativo, no una garantía estadística universal.

Ejemplo real: ATM-TX-00034

Su monto original es null. El grupo PAGO_SERVICIOS con FALLO_FONDOS tiene **19 observaciones**, mediana **4,000.00** y media aproximada **5,894.74**. Se asignan 4,000.00 MXN y se registra MEDIANA_GRUPO.

La mediana es el centro al ordenar. La media suma y divide por la cantidad. Ejemplo didáctico: 100, 100, 200, 300, 10,000 tienen mediana 200 y media 2,140. El extremo mueve mucho la media; por eso preferimos mediana para montos sesgados.

> Consulta de saldo faltante = 0; cajero faltante = DESCONOCIDO; concepto faltante = SIN_CONCEPTO. Algunas etiquetas ya se aplican en los validadores de Leo. Se conserva su trazabilidad y no se obliga a inventar correos opcionales.

3. IQR y monto analítico

Outlier es un valor extremo respecto a su grupo. Puede ser válido, por eso no se elimina solo por ser alto. **Analítico** significa preparado para comparar y calcular estadísticas; no es un nuevo importe bancario.

```
IQR = Q3 - Q1
límite inferior = max(0, Q1 - 1.5 × IQR)
límite superior = Q3 + 1.5 × IQR
analítico = min(superior, max(inferior, monto_mxn))
```

Q1 y Q3 son los percentiles 25 y 75. Se calculan con interpolación lineal. Los límites se ajustan por fuente y operación usando montos observados válidos. Se excluyen consultas de saldo y montos imputados con mediana del ajuste.

Ejemplo real DIGITAL	Valor MXN
Q1 / Q3	12,041.145 / 34,946.655
IQR	22,905.510
Límite superior	69,304.920
SPEI-2026-00003: monto de operación	552,091.22
SPEI-2026-00003: monto analítico	69,304.92

El registro conserva monto_mxn y genera monto_analitico_mxn limitado. El segundo se usa para MinMax y segmentos. es_outlier queda true. No debes usar el monto limitado como si fuera el dinero de la operación.

Si hay menos de cinco observaciones o IQR cero, no se aplica recorte y se guarda el motivo. La entrega tiene **119 registros marcados como outliers**. La cifra cuenta filas, no dinero.

> Un dato imposible se rechaza según validación. Cero es válido para consulta de saldo y comisión; las operaciones monetarias requieren monto positivo. Un inválido y un extremo estadístico son cosas diferentes.

4. Seguir el código y practicar

Función	Entrada y salida
perfilar	Crudos; devuelve métricas sin modificar la entrada.
separar_exactos	Crudos; devuelve únicos y omitidos con posiciones.
imputar	Candidatos normalizados; devuelve candidatos, rechazados y parámetros.
aplicar_iqr	Candidatos imputados; añade monto analítico, outlier y límites.
comparar_calidad	Perfiles antes/después y conteos; produce evidencia.

Avilix conecta tus funciones en pipeline.py. Antes de ajustar estadísticas se validan formatos, separan conflictos y cruzan cuentas. Después de tu capping, Avilix calcula MinMax y segmentos; Leo persiste. Así no se ajustan estadísticas con montos corruptos o huérfanos.

Práctica desde la raíz del repositorio

```
.venv\Scripts\python.exe entrega/guias/consultar_ejemplos.py maki
```

1. Busca la mediana del grupo PAGO_SERVICIOS y FALLO_FONDOS: tamaño 19, mediana 4000 y media 5894.74.
2. En ejemplo_imputacion, identifica el null y el valor asignado.
3. En ejemplo_iqr, resta Q3 − Q1 y calcula el límite superior.
4. Explica por qué ambos montos siguen guardados.

```
.venv\Scripts\python.exe -m pytest -q tests/test_calidad.py
.venv\Scripts\python.exe -m pytest -q tests/test_limpieza.py
```

El consultor abre la base en modo de solo lectura. Las pruebas usan casos controlados. Las 50 aprobadas corresponden a la batería completa registrada, no a cada comando individual.

Dónde está la evidencia

entrega/resultados.json guarda perfiles, comparacion y parametros por ejecución. SQLite guarda evidencia_lotes y cambios por registro. docs/reporte_maki.md es un aporte previo; usa la evidencia integrada para cifras finales y trata MCAR/MAR como hipótesis, sin afirmar causalidad probada.

Cómo explicarlo con tus palabras

Una explicación breve

“Mi parte mide calidad, separa copias exactas y trata faltantes con mediana por grupo y un mínimo de cinco observaciones. Para extremos calculo IQR con factor 1.5 y limito solo el monto analítico. Guardo parámetros y cambios para explicar el resultado.”

Preguntas que debes poder responder

¿Por qué mediana?
Es menos sensible a extremos que la media.

¿Recuperaste el monto verdadero?
No. Se estimó con una regla documentada y se conserva el faltante original.

¿Capping elimina filas?
No. Limita la variable analítica y conserva el importe de operación.

¿Demostramos MAR?
No. Es una hipótesis operativa; una conclusión más fuerte exige análisis adicional.

Comprueba que lo entiendes

Explica un ejemplo sin leer, señala la función que lo realiza y encuentra su resultado en la base. Distingue un dato observado, una estimación y una regla de negocio. Si puedes ejecutar el ejemplo y justificarlo, ya tienes una base para defenderlo.

Fuentes de esta guía

Guía del profesor: GUIA_LINEAMIENTOS_PROYECTO_ALUMNOS.pdf, puntos 4 y 6, páginas 2 y 3. Código: src/calidad.py, src/duplicados.py, src/limpieza.py, src/pipeline.py. Evidencia: entrega/pipeline_banco.sqlite, entrega/resultados.json y entrega/pruebas.txt. Las cifras corresponden a la base entregable original.